

I couldnt find a archive of intersection footage, which might be a problem

Given the data I found, we could...:

- Tabular data — > predicts a risk score that is outputted on a heatmap on our app/website . (How many crashes should we expect per year?)
- Cctv data: —> live updates of crashes + more

More links to Datasets: !! :

Pls paste:

The data i found below is all from : data-seattlecitygis.opendata.arcgis.com – (although we ill use their api for the following:

Potential predictors we could use for the risk model

Camera locations (ArcGIS: Traffic_Cameras)

- Lat/Ing, camera ID, name, ownership (SDOT vs WSDOT)
- Static metadata only, refreshed weekly

Live camera frames (SDOT HLS streams, not ArcGIS)

- Public video streams, roughly 10 second chunks
- One frame per 30 seconds per camera, pulled by your scraper
- Not archived by SDOT, so your scraper is the storage

Crashes (ArcGIS: Collisions)

- Lat/Ing, datetime, severity, mode (vehicle, pedestrian, bike), contributing factors
- 2003 to present, around 150k records, refreshed regularly

Traffic volume / AADT (ArcGIS: {year}_Traffic_Flow_Counts)

- Annual average daily traffic per road segment
- One service per year from 2007 onward
- Used as the exposure denominator in the risk model

Signals and infrastructure (ArcGIS: SDOT_Assets MapServer)

- Traffic signal locations, signs, signal cabinets
- Layer 9 is cameras, other layers cover related assets

Bike infrastructure (ArcGIS: Bike_Facilities)

- Bike lane geometry by street segment
- Used as a feature in the risk model

Speed limits (ArcGIS: Speed_Limits)

- Posted speed by street segment
- Used as a feature in the risk model

Enforcement cameras (ArcGIS: Automatic_Traffic_Safety_Cameras)

- Red light, school zone speed, bus lane, blocking-the-box cameras
- Different from monitoring cameras
- Useful as a map overlay, not a model input

Census layers (ArcGIS: 2020_Census_Tracts_Seattle, etc.)

- Block groups and tracts, geometry only
- Join to ACS demographics (income, race, vehicle access) via Census API
- Used for the equity analysis layer

Intersection geometry (OpenStreetMap via Overpass API)

- Intersection nodes, approach counts, lane counts where SDOT data is missing
- Fallback only, since most features now come from SDOT directly

idea (we can change this if yall want) is that the tabular data answers one question: given an intersection's physical features and how busy it is, how many crashes should we expect there per year? (all from ArcGIS portal)

And then The cctv data answers a different question of what is actually happening at each intersection right now?

The super ambitious we could use the Claude vision model watches frames scraped SDOT cameras and flags safety relevant events (near-misses, illegal turns, blocked crosswalks, pedestrian conflicts). Each flagged event becomes a row in the in our database , tagged with the camera, the intersection address , and the event type.

I couldnt find a archive of intersection footage, which might be a problem

Video pipeline (always running)

A Python script pulls one frame every 30 seconds from each SDOT camera. Frames go to Supabase Storage.

YOLO checks each frame. Is anything moving? If no, drop the frame. If yes, send it to Claude.

Claude looks at the frame and returns JSON: what event happened, who was involved, how severe, what countermeasure to suggest. The result gets written to a `vlm_events` table.

Over time, each intersection builds up a count of flagged events.

Tabular pipeline

A second script pulls every Seattle dataset from the ArcGIS portal: cameras, crashes, AADT, signals, bike lanes, speed limits, census. All of it goes into Postgres.

Then we run spatial joins to build the intersections table. One row per intersection. Columns for every feature: signal type, lane count, bike lane present, speed limit, traffic volume, historical crash count.

This runs once at project start. Refresh monthly.

the risk model

Fit a risk model (expected value) on the tabular data. It learns to predict expected crashes per year from infrastructure and traffic volume. No video involved yet.

For every intersection, we now have an expected baseline.

The video pipeline gives us an observed event rate per intersection. The risk score could look like this:

$$\text{risk_score} = (\text{observed_rate} - \text{expected_rate}) / \text{sqrt}(\text{expected_rate})$$

Above zero: the intersection has more conflicts than its features predict. Below zero: fewer. Score gets written to `intersection_scores`, refreshed weekly.

Frontend reads the merged result

The map pulls from `intersection_scores` and colors pins by score. Click a pin and the drill in panel runs four lookups for that `intersection_id`: crashes for the timeline, `vlm_events` for recent clips, intersections for the metadata, and a static lookup that maps the dominant event type to a countermeasure.

Very rough outline of project....

- There most likely will be other scripts that we will write.

```
seattle-conflict-watch/
├── .env.example
├── frontend/
│   ├── package.json
│   ├── app/
│   │   └── page.tsx
│   └── components/
│       ├── Map.tsx
│       └── DrillInPanel.tsx
├── db/
│   └── schema.sql
└── pipeline/
    ├── requirements.txt
    ├── fetch_data.py
    ├── scrape_frames.py
    ├── detect_events.py
    ├── score_risk.py
    └── db.py
```

- `.env.example` — Supabase URL, Claude key.

frontend

- `app/page.tsx` — the map page.
- `components/Map.tsx` — MapLibre map with pins colored by risk score.
- `components/DrillInPanel.tsx` — clicked-intersection sidebar with crash history and recent events.

db

- `schema.sql` — all tables (cameras, frames, vlm_events, intersections, crashes, intersection_scores) in one file.

pipeline

- `db.py` — Supabase client, insert/query helpers.
- `fetch_data.py` — pulls cameras, crashes, AADT, etc. from Seattle's ArcGIS into the DB.
- `scrape_frames.py` — ffmpeg loop pulling SDOT frames every 30s.
- `detect_events.py` — sends frames to Gemini Flash, parses JSON, writes events to DB.
- `score_risk.py` — fits the negative binomial baseline and writes risk scores per intersection.